Modern CSS Layout & Responsive Design

Flexbox • CSS Grid • Responsive Design • Bootstrap • Tailwind CSS

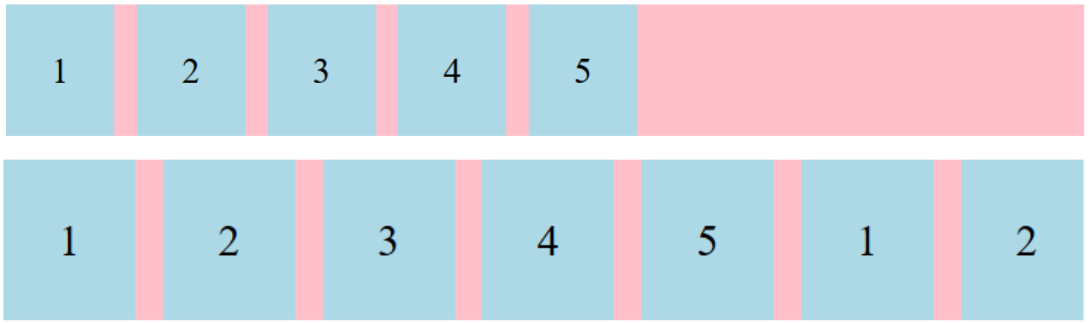
Web Technologies

Comsats University Islamabad, Attock Campus

CSS Layout Designs

- There are two different techniques to create multicolumn layouts.
  - CSS flexbox
  - CSS grid

CSS Flexbox




```
.container {
            display: flex;
            flex-direction: row;
            background-color: pink;
            gap: 10px;
        }
        .container div {
            background-color: lightblue;
            padding: 20px;
            text-align: center;


        }
```

- CSS Flexible Box Layout, commonly known as Flexbox, is a CSS web layout model.
- The flex layout allows responsive elements within a container to be automatically arranged depending on the viewport (device screen) size.
- Vertically center a block of content inside its parent.
- Make all the children of a container take up an equal amount of the available width/height, regardless of how much width/height is available.
- Make all columns in a multi-column layout adopt the same height even if they contain a different amount of content.

CSS Flexbox

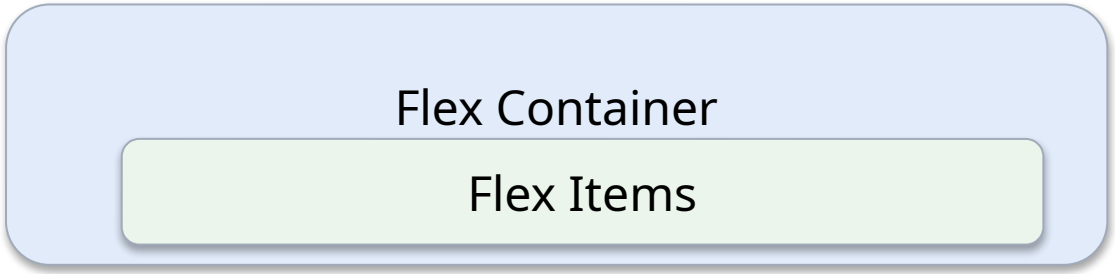
Flexbox is primarily a one-dimensional layout system.

Flex Container

Created by display: flex;

Direct children become flex items.

Controls direction, spacing, alignment and wrapping.



Direct children of the flex container.

Can grow, shrink and have a basis.

Can individually override alignment with align-self.

Flex Container and Flex Items

Parent = flex container.

Direct children = flex items.

Nested grandchildren are not flex items of the outer container.

Flexbox starts with the main axis and cross axis.

```
.container
    │
    ├── Item 1  ← Flex item
    ├── Item 2  ← Flex item
    └── Item 3  ← Flex item
```

```
.container {
  display: flex;
}

.container > div {
  padding: 20px;
  border: 1px solid #999;
}
```

Main Axis vs Cross Axis

Main Axis

Direction in which flex items are laid out.

Controlled by flex-direction.

row → left to right by default.

column → top to bottom.

```
.container {
            display: flex;
            flex-direction: row;
}
```

justify-content works on the **main axis**.

align-items works on the **cross axis**.

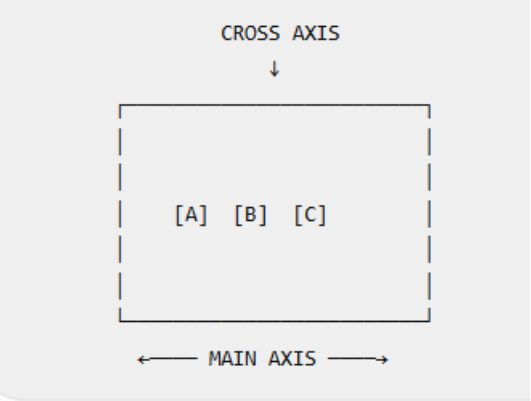
Cross Axis

Axis perpendicular to the main axis.

align-items controls cross-axis alignment.

For row, cross axis is vertical.

For column, cross axis is horizontal.



flex-direction

row — horizontal, left → right (default).

row-reverse — horizontal, right → left.

column — vertical, top → bottom.

column-reverse — vertical, bottom → top.

Changing direction changes the meaning of the main axis.

```
.container {
  display: flex;
  flex-direction: row;
}

/* Other values */
flex-direction: row-reverse;
flex-direction: column;
flex-direction: column-reverse;
```

justify-content: Main-Axis Alignment

Works along the main axis.

flex-start — pack at the start.

center — pack in the center.

flex-end — pack at the end.

space-between — equal gaps between items.

space-around / space-evenly — distribute free space.

```
.container {
  display: flex;
  justify-content: space-between;
}

/* common values */
flex-start;
center;
flex-end;
space-between;
space-around;
space-evenly;
```

align-items: Cross-Axis Alignment

Works along the cross axis.

In a row layout, it normally controls vertical alignment.

center is useful for vertically centering content.

stretch is the default when item cross-size is auto.

Do not confuse align-items with justify-content.

```
.container {
  display: flex;
  height: 250px;
  align-items: center;
}

/* common values */
stretch;
flex-start;
center;
flex-end;
baseline;
```

flex-wrap: Handling Extra Items

nowrap keeps items on one line and may cause overflow.

wrap moves items to additional lines when needed.

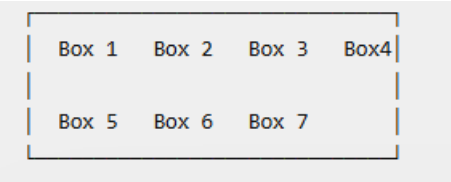
wrap-reverse reverses the cross-axis line direction.

Use with gap for clean card layouts.

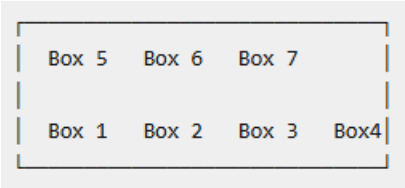
Flex-wrap is especially useful for responsive navigation and cards.

```
.container {
  display: flex;
  flex-wrap: wrap;
  gap: 16px;
}

/* values */
nowrap;   /* default */
wrap;
wrap-reverse;
```



wrap



wrap-reverse

Flex Item Sizing: grow, shrink and basis

flex-basis — starting/main-size before free space is distributed.

flex-grow — how much an item can receive from extra space.

flex-shrink — how much an item can shrink when space is insufficient.

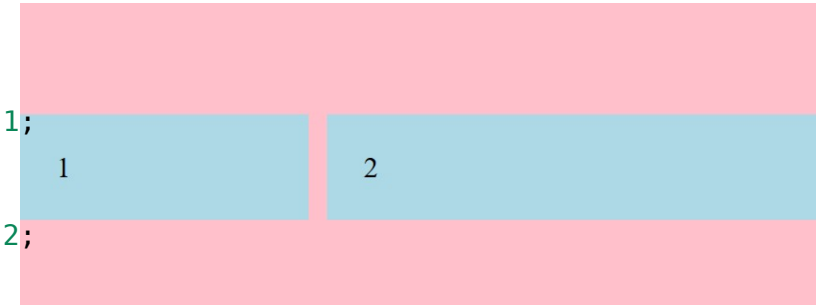
flex: grow shrink basis is the shorthand.

flex: 1 is a common pattern for equal flexible columns.

```
.item {
  flex-grow: 1;
  flex-shrink: 1;
  flex-basis: 200px;
}

/* shorthand */
flex: 1 1 200px;
```

```
.b1 {
        flex-grow: 1;
    }
 .b2 {
        flex-grow: 2;
    }
```



A Practical Flexbox Navigation Bar

Use Flexbox when items mainly form a row or column.

justify-content distributes links horizontally.

align-items vertically centers them.

gap creates consistent spacing without margin hacks.

```
<nav class="nav">
  <a>Home</a>
  <a>Courses</a>
  <a>Labs</a>
  <a>Contact</a>
</nav>

.nav {
  display: flex;
  justify-content: space-between;
  align-items: center;
  gap: 20px;
}
```

1	2	3
4	5	

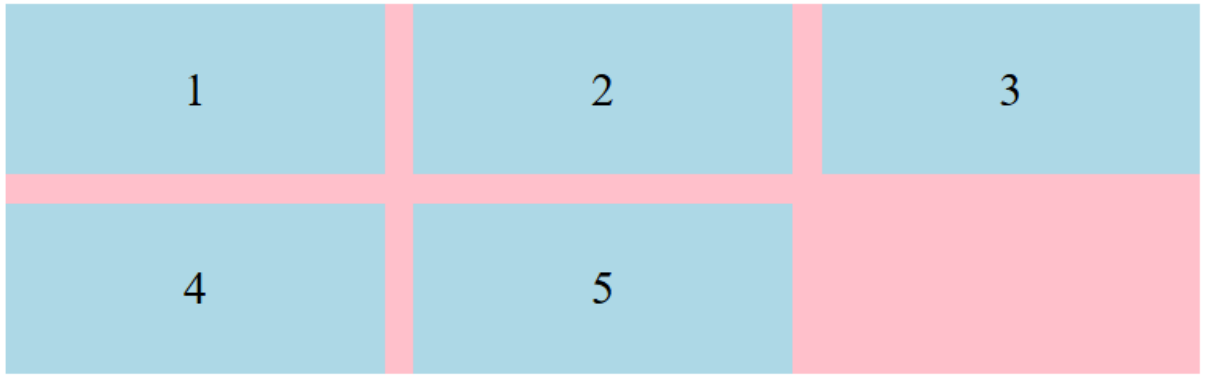
CSS Grid

- Like tables, grid layout enables an author to align elements into columns and rows.
- However, many more layouts are either possible or easier with CSS Grid than they were with tables.
- The grid layout is responsive and flexible, adjusting to different screen sizes.

CSS Grid

```html
<div class="container">
        <div>1</div>
        <div>2</div>
        <div>3</div>
        <div>4</div>
        <div>5</div>
    </div>
```

```css
.container {
        display: grid;
        grid-template-columns: repeat(3, 1fr);
        gap: 10px;
        background-color: pink;
    }
    .container div {
        background-color: lightblue;
        padding: 20px;
        text-align: center;
    }
```



1fr = one equal share (fraction) of the available grid space.

2. CSS Grid: Two-Dimensional Layout

Grid controls rows AND columns simultaneously.

Grid Container

Created by display: grid;

Defines columns, rows, gaps and placement.

Direct children become grid items.

```css
.grid{
    display:grid;
    grid-template-rows: 100px 100px;
    grid-template-columns: repeat(5,1fr);
    background-color: lightgreen;

}
```

Grid Concepts

Tracks = rows or columns.

Lines = boundaries around tracks.

Cells = one row/column intersection.

Areas = named groups of cells.

Concept	Meaning
Grid container	Parent with `display: grid`
Grid item	Direct child of grid container
Rows	Horizontal tracks
Columns	Vertical tracks
Grid lines	Boundaries used for positioning
Grid cell	Smallest grid unit
Grid track	A row or column
Gap	Space between tracks
`fr`	Fraction of available space
Spanning	Item occupying multiple rows/columns
Grid area	Rectangular region of cells

grid-template-columns, rows and fr

grid-template-columns defines column tracks.

grid-template-rows defines row tracks.

fr means a fraction of available grid space.

1fr 2fr 1fr gives the middle column twice the flexible space.

auto sizes according to content and available space.

```
.grid {
  display: grid;
  grid-template-columns: 1fr 2fr 1fr;
  grid-template-rows: 100px auto;
  gap: 20px;
}
```

Grid gap and the Grid Box Model

gap creates space between grid tracks.

row-gap controls vertical spacing.

column-gap controls horizontal spacing.

repeat(3, 1fr) creates three equal columns.

Unlike margins, gap is specifically the spacing between grid/flex items.

```
.grid {
  display: grid;
  grid-template-columns: repeat(3, 1fr);
  gap: 24px;
  /* row-gap: 24px;
     column-gap: 24px; */
}
```

Grid Line Placement

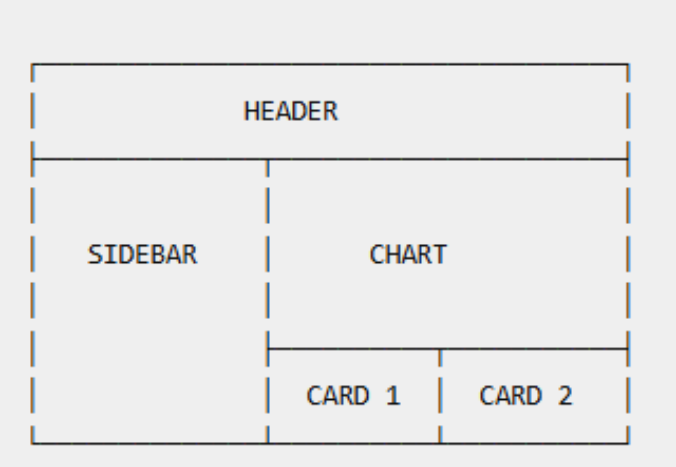
Grid lines are numbered from the start edge.

grid-column: 1 / 3 occupies columns between lines 1 and 3.

grid-row: 2 / 4 occupies rows between lines 2 and 4.

span 2 means occupy two tracks from the starting position.

Placement is useful for dashboards and magazine-style layouts.




```
.item {
  grid-column: 1 / 3;
  grid-row: 2 / 4;
}

/* span syntax */
.item {
  grid-column: span 2;
}
```

Named Grid Areas

grid-template-areas gives semantic names to regions.

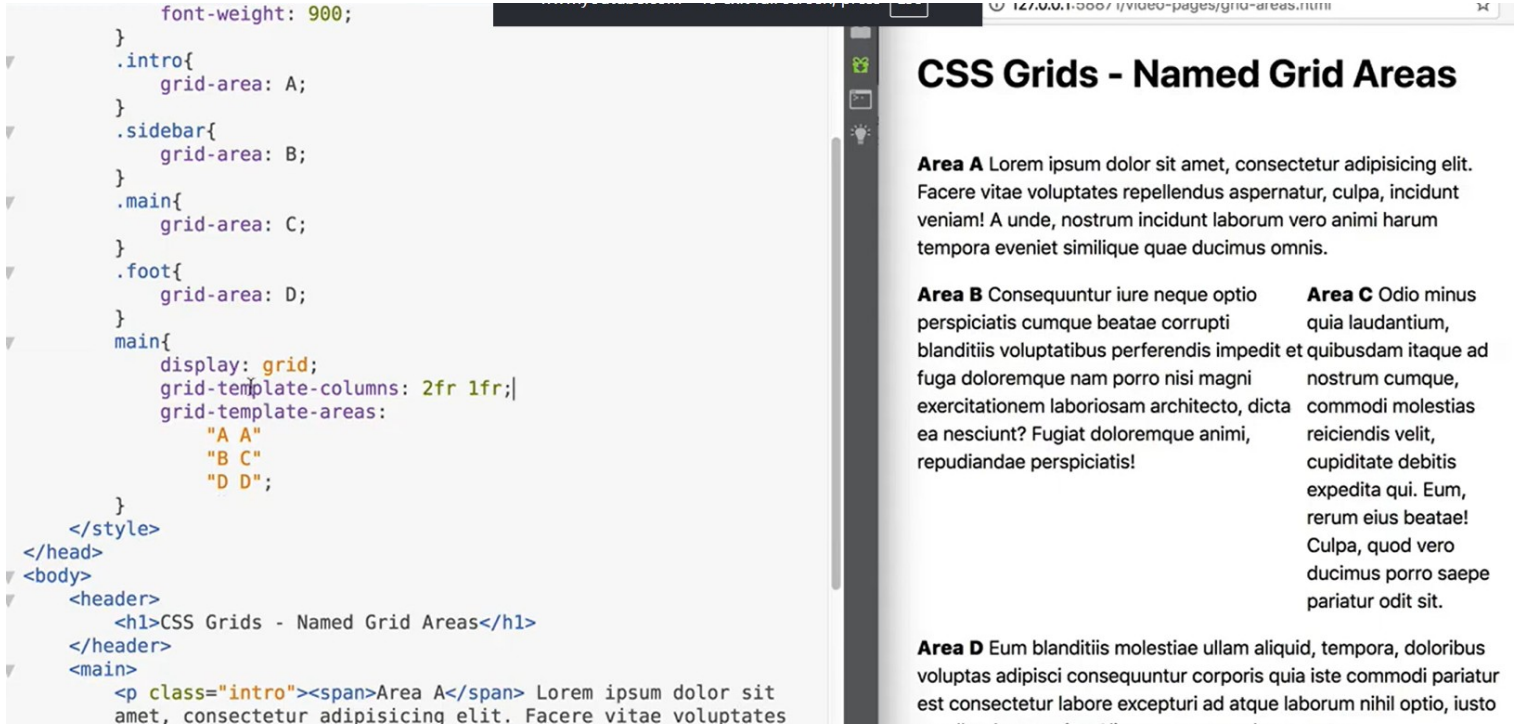
Repeated names span multiple cells.

A dot (.) can represent an empty cell.

Makes complex page structure easier to read and maintain.

```
.page {
  display: grid;
  grid-template-areas:
    "header header"
    "sidebar main"
    "footer footer";
}

.header {
grid-area: header; }
.sidebar {
 grid-area: sidebar; }
.main { grid-area: main; }
.footer { grid-area: footer; }
```



Flexbox vs CSS Grid: Which Should You Choose?

Choose Flexbox when...

The layout is mainly one-dimensional.

You are aligning items in a row or column.

Content size should naturally influence spacing.

Examples: navigation, button groups, card internals.

Choose Grid when...

Rows and columns matter together.

You need explicit placement or spanning.

You are building a page/dashboard structure.

Examples: gallery, dashboard, page shell.

They can be combined: Grid for page structure + Flexbox inside components.

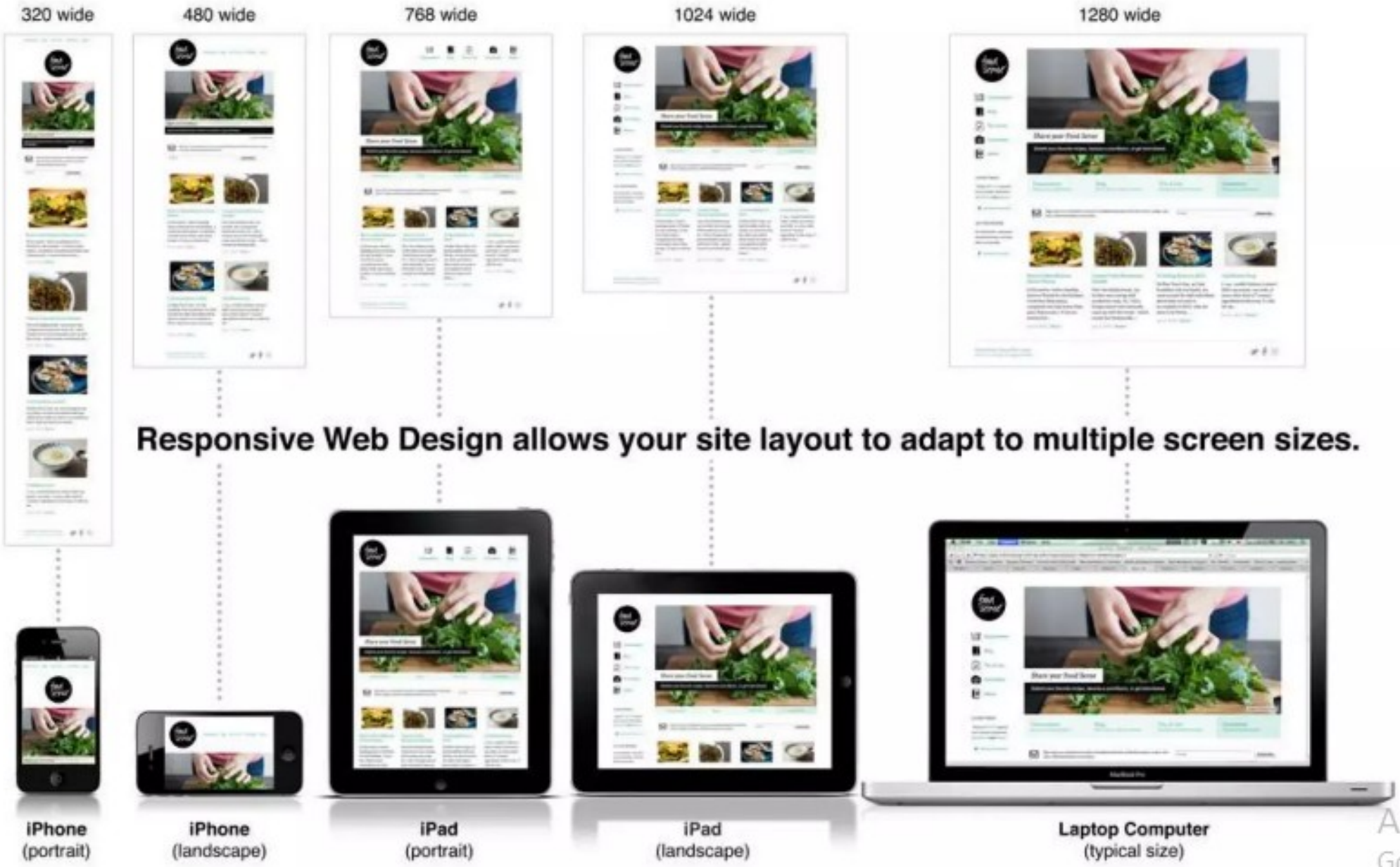
Responsive Web Design



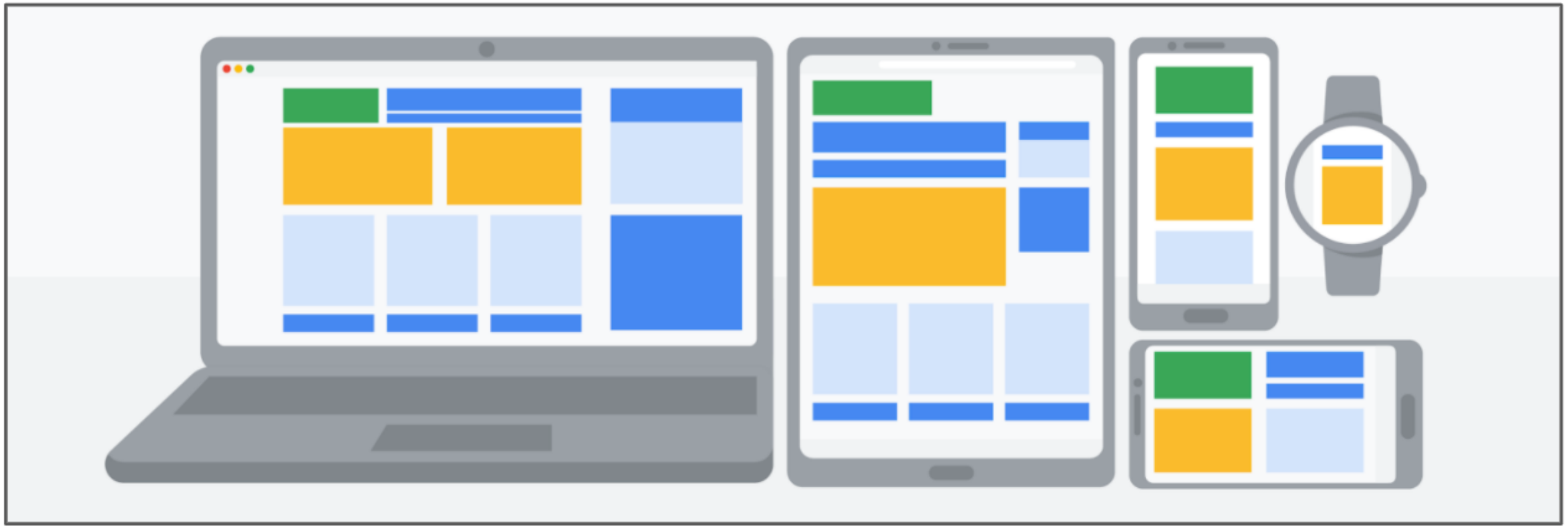
"**Responsive web design** (**RWD**) is a web design approach aimed at crafting sites to provide an optimal viewing experience – easy reading and navigation with a minimum of resizing, panning, and scrolling – across a wide range of devices (from mobile phones to desktop computer monitors), based on screen size, platform and orientation. True responsive design is fluid, using CSS3 media queries to respond to any screen sizes. With the use of this CSS3 module, you can create a flexible grid where text can wrap and images can shrink to adjust along with your browser."

Activate Windo
Go to Settings to ac

Responsive Web Design



Responsive Layout



How responsive design works?

Responsive design is client side, meaning that the whole webpage is delivered to the device browser (the client), and the layout is adjusted to respond to the screen display.



RESPONSIVE

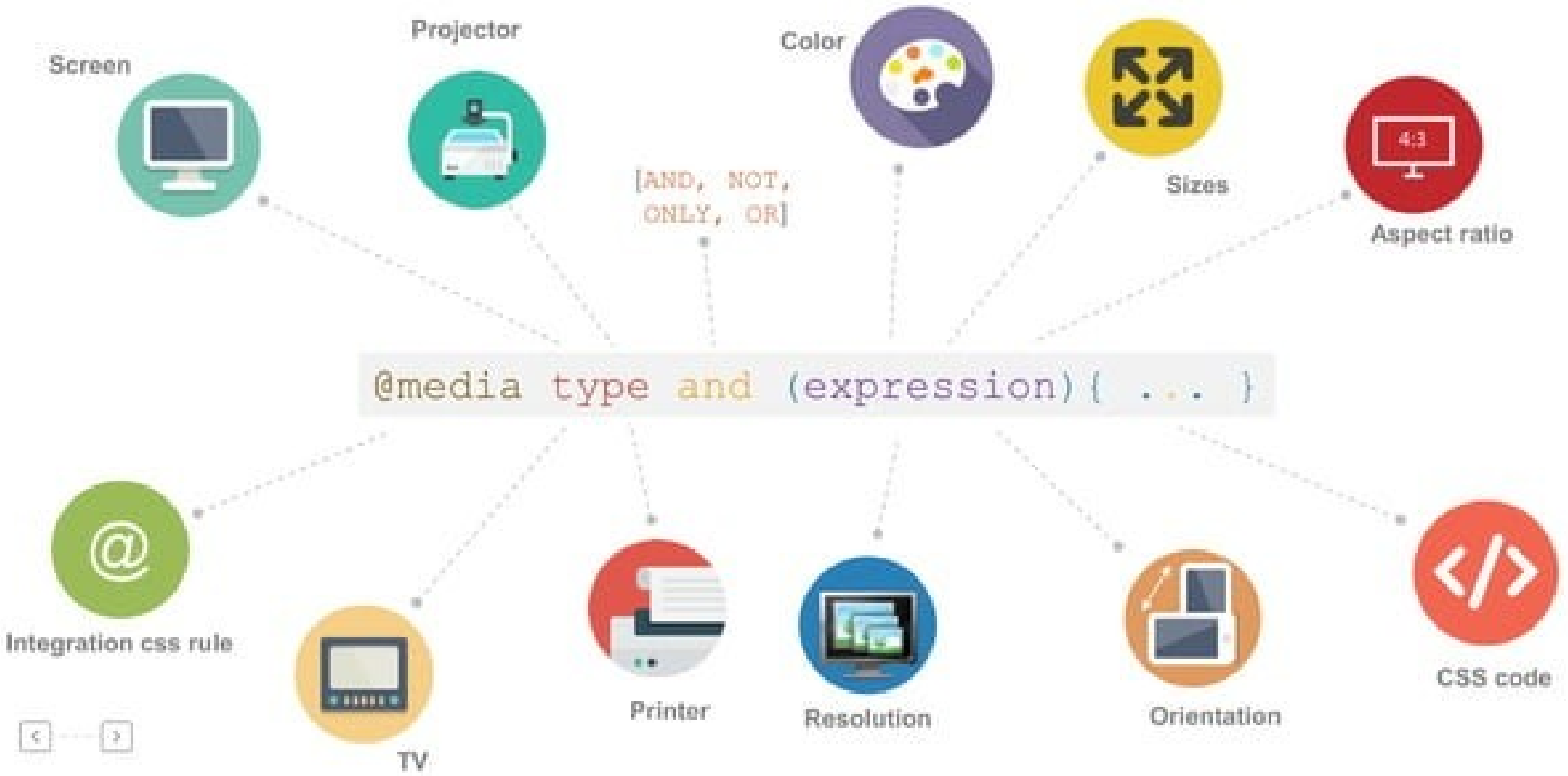
HOW TO

Media Queries

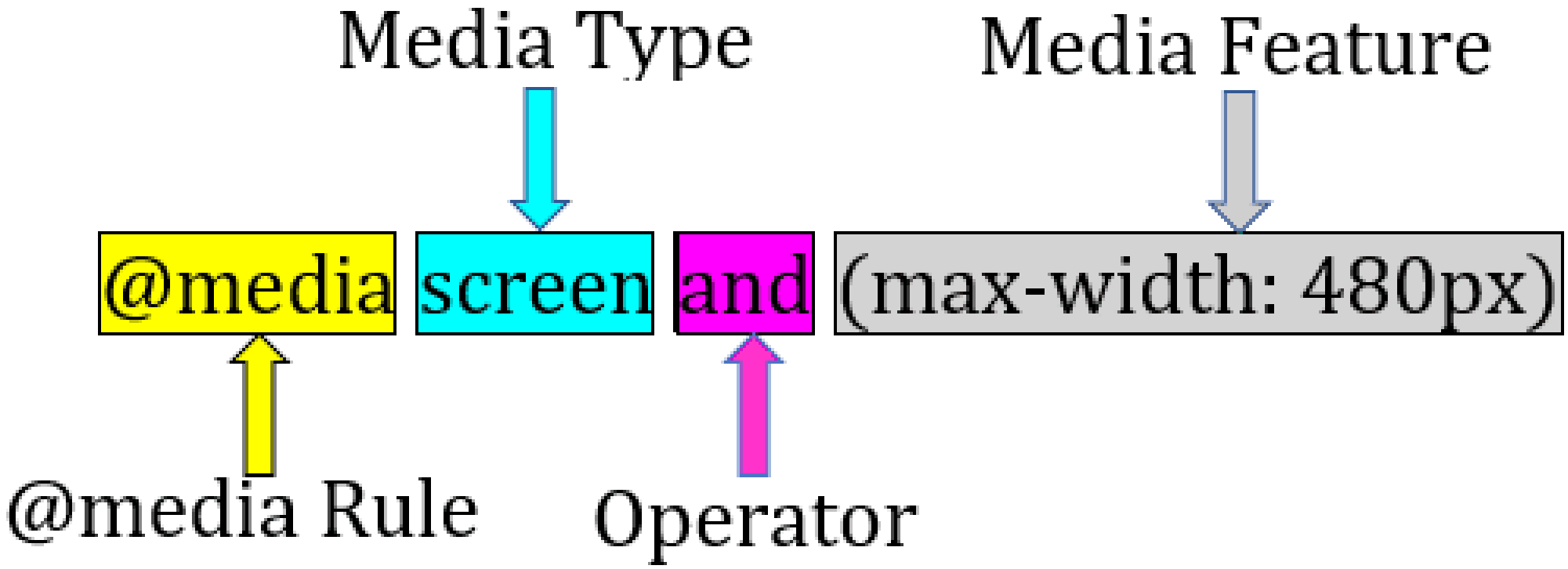
•Media queries are a popular technique for delivering a tailored style sheet to different devices. Media queries allow you to apply CSS styles depending on:

- width and height of the viewport
- width and height of the device
- Orientation (is the tablet/phone in landscape or portrait mode?)
- Resolution

Media Query Syntax



Media Query


Media Type
Media Feature
@media
screen
and
(max-width: 480px)
@media Rule
Operator

Media Query with and operator

```
@media screen and (min-width: 480px) {

  #leftsidebar {width: 200px; float: left;}

  #main {margin-left: 216px;}

}
```

Media Query with or operator

```
@media (max-width: 599px), (min-width: 901px) {
    body {
        background-color: lightcoral;
    }
}
```

Media Query with not operator

```
@media not (min-width: 600px) {
    body {
        background-color: lightgreen;
    }
}
```

Mobile-First Design

Write the base CSS for smaller screens first.

Use min-width media queries to add layout complexity.

Advantages: simpler base styles and better progressive enhancement.

Do not assume every mobile device is exactly the same width.

```
/* Base = small screen */
.card-grid {
  display: grid;
  grid-template-columns: 1fr;
}

/* Larger screens */
@media (min-width: 768px) {
  .card-grid {
    grid-template-columns: repeat(2, 1fr);
  }
}
```

Viewport Meta Tag

Put this inside <head>.

width=device-width makes the CSS viewport match the device width.

initial-scale=1.0 sets the initial zoom level.

Essential for responsive pages on mobile browsers.

```
<meta name="viewport"
      content="width=device-width,
               initial-scale=1.0">
```

Fluid Units: %, rem, vw and vh

Relative to what?

% — often relative to the containing block/property context.

rem — relative to the root element's font size.

vw — 1% of viewport width.

vh — 1% of viewport height.

Practical Examples

width: 80%;

padding: 1rem;

font-size: 4vw;

min-height: 100vh;

Prefer flexible units where fixed pixels would cause overflow.

Feature Queries

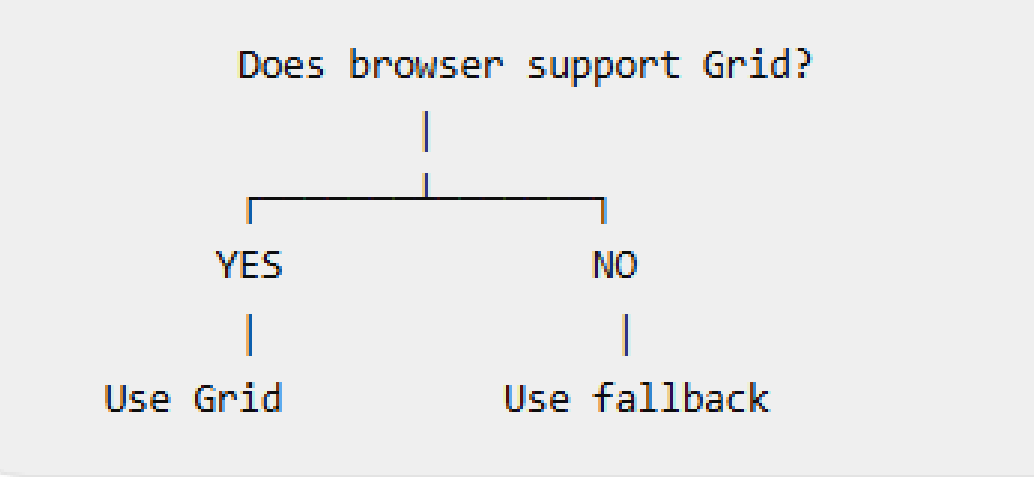
Feature queries use @supports.

They check whether the browser supports a particular CSS feature.

They can be used to provide fallbacks for newer CSS features.

```
.container {
    display: block;
}

@supports (display: grid) {
    .container {
        display: grid;
        justify-content: space-between;
    }
}
```



Responsive Images

max-width: 100% prevents an image from overflowing its container.

height: auto preserves aspect ratio.

<picture> can serve different image files for different conditions.

Use meaningful alt text for informative images.

For performance, also consider appropriately sized/compressed images.

```
img {
  max-width: 100%;
  height: auto;
}

<picture>
  <source media="(min-width: 900px)"
          srcset="large.jpg">
  <img src="small.jpg"
       alt="Product image">
</picture>
```

CSS Framework Fundamentals

Bootstrap and Tailwind solve common UI/layout problems with different philosophies.

Bootstrap

Component-based framework.

Provides grid, buttons, forms, cards, navbar, modal, utilities.

Convention-heavy: predefined class patterns.

Good for rapidly building conventional interfaces.

Tailwind CSS

Utility-first CSS framework.

Compose small classes directly in markup.

Responsive variants are built into class names.

Highly customizable through configuration/theme.

Bootstrap → *"Give me a ready-made button."*
Tailwind → *"Give me the tools to design my own button."*

Component-Based vs Utility-First

Component-Based

Think in reusable UI components.

Example: btn, card, navbar, modal.

Framework provides ready-made visual patterns.

Often faster for standard business interfaces.

btn btn-primary

Primary style

Utility-First

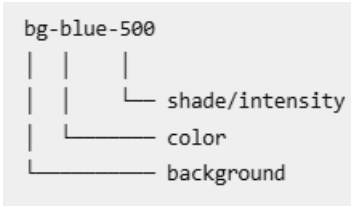
Think in small styling utilities.

Example: flex, p-4, text-center, rounded-lg.

Compose the exact design in markup.

Reduces the need for many custom component CSS rules.

bg-blue-500 text-white px-6 py-3



bg-blue-500 → background color
text-white → text color
px-6 → horizontal padding
py-3 → vertical padding

Bootstrap: Grid System

Bootstrap uses a responsive 12-column grid concept.

container provides a responsive content wrapper.

row groups columns.

col-12 = full width on the smallest breakpoint.

col-md-6 = half width from the md breakpoint upward.

Columns can be combined to form responsive layouts.

```
<div class="container">
  <div class="row">
    <div class="col-12 col-md-6">
      A
    </div>
    <div class="col-12 col-md-6">
      B
    </div>
  </div>
</div>
```

Bootstrap Components

Components are pre-styled UI building blocks.

Common examples: buttons, cards, navbar, forms, alerts, modal.

Combine component classes with utility classes.

Always check the framework version because class behavior can change.

```
<button class="btn btn-primary">
  Save
</button>

<div class="card">
  <div class="card-body">
    <h5 class="card-title">Course</h5>
    <p class="card-text">Web
Technologies</p>
  </div>
</div>
```

Bootstrap Utilities

Utilities provide small, focused styling classes.

d-flex → display:flex

justify-content-between → distribute items.

align-items-center → cross-axis centering.

p-3 → padding; mb-3 → margin-bottom.

Utilities reduce the need for custom CSS for common tasks.

```
<div class="d-flex
            justify-content-between
            align-items-center
            p-3
            mb-3">
  <span>Title</span>
  <button class="btn btn-primary">
    Action
  </button>
</div>
```